Loan Approval Prediction System

Using Machine Learning (Logistic Regression & XGBoost)

Intro to GenAI Capstone Project (Milestone 1)

Newton School of Technology, Sonipat

Team Members

Mitul Bhatia	Team Lead, ML Development
Hardik	System Architecture
Vaibhav	Documentation
Sarvjeet	Presentation & Video

Project Links

GitHub: https://github.com/mitul-bhatia/GEN_AI_LOAN_APPROVAL
Live Demo: https://genailoanapproval-mitul.streamlit.app/

Final Accuracy: 92.86% | ROC-AUC: 98.42%

February 2026

Contents

1 Introduction **4**
- 1.1 Problem Statement . . . 4
- 1.2 Objectives . . . 4
- 1.3 Scope . . . 4

2 Dataset Description **5**
- 2.1 Overview . . . 5
- 2.2 Features . . . 5
- 2.3 Target Variable Distribution . . . 5

3 Exploratory Data Analysis **7**
- 3.1 Numerical Features Distribution . . . 7
- 3.2 Categorical Features Analysis . . . 7
- 3.3 Correlation Analysis . . . 8
- 3.4 Key Features Deep Dive . . . 10

4 Data Preprocessing **12**
- 4.1 Feature Selection . . . 12
- 4.2 Categorical Encoding . . . 12
- 4.3 Feature Scaling . . . 12
- 4.4 Train-Test Split . . . 12

5 Model 1: Logistic Regression **13**
- 5.1 Why Logistic Regression? . . . 13
- 5.2 Mathematical Foundation . . . 13
  - 5.2.1 Loss Function . . . 13
  - 5.2.2 Regularization . . . 13
- 5.3 Hyperparameter Tuning . . . 14
- 5.4 Results . . . 14
  - 5.4.1 Classification Report . . . 14
  - 5.4.2 Cross-Validation Stability . . . 14
- 5.5 Confusion Matrix and ROC Curve . . . 15
- 5.6 Feature Importance Analysis . . . 16
  - 5.6.1 Coefficient Interpretation . . . 16
- 5.7 Key Insights . . . 17
- 5.8 Conclusion for Logistic Regression . . . 17

6 Error Analysis: Why 86%? **18**
- 6.1 Error Breakdown . . . 18
- 6.2 Key Finding: Model is Confidently Wrong . . . 18
- 6.3 Key Finding: Feature Interactions Matter . . . 19
- 6.4 Root Cause Analysis . . . 20

7 Model 2: XGBoost **21**
- 7.1 Why XGBoost After Logistic Regression? . . . 21
- 7.2 Mathematical Foundation . . . 21
  - 7.2.1 From Single Tree to Gradient Boosting . . . 21

7.2.2 Objective Function . . . 21
7.2.3 Regularization . . . 21
7.3 Hyperparameter Tuning . . . 22
7.4 Results . . . 22
7.5 Error Analysis: What XGBoost Fixed . . . 23
7.6 Feature Importance . . . 23

8 Streamlit Web Application **25**
8.1 System Architecture . . . 25
8.2 Application Components . . . 25
8.3 User Interface Features . . . 26
8.4 Deployment . . . 26
8.5 Technology Stack . . . 26

9 Summary and Conclusions **27**
9.1 Work Completed . . . 27
9.2 Key Insights . . . 27
9.3 Final Model Performance . . . 27

10 Team Contribution **28**
10.1 Contribution Matrix . . . 28
10.2 Individual Contributions . . . 28
10.3 Code Ownership Declaration . . . 29
10.4 Dataset Source . . . 30

Abstract

This report presents a comprehensive machine learning approach to predict loan approval decisions. Using a dataset of 50,000 loan applications with 20 features, we develop and evaluate multiple classification models. Our analysis begins with Logistic Regression as a principled baseline, achieving 86.42% accuracy and 0.944 ROC-AUC. Error analysis revealed that feature interactions (credit score × DTI) explain 40% of classification errors, motivating the use of XGBoost. The final XGBoost model achieves **92.86% accuracy** and **0.984 ROC-AUC**, a +7.45% improvement by capturing non-linear interactions. The model is deployed as a production Streamlit application. Key findings reveal that product type and loan intent are stronger predictors than traditional credit metrics, with Credit Card products showing 33x higher approval odds compared to baseline.

1 Introduction

1.1 Problem Statement

Loan approval is a critical decision in financial institutions, balancing risk management with customer acquisition. Manual assessment is time-consuming, inconsistent, and cannot scale to handle millions of applications. This project develops an automated loan approval prediction system using machine learning.

1.2 Objectives

1. Develop accurate classification models to predict loan approval
2. Identify key factors influencing approval decisions
3. Compare traditional ML algorithms with deep learning approaches
4. Deploy a production-ready web application

1.3 Scope

This project implements:

- Exploratory Data Analysis (EDA) with 6 comprehensive visualizations
- Baseline Model: Logistic Regression with full mathematical foundation (86.42% accuracy)
- Error Analysis: Diagnosing why the baseline model fails
- Production Model: XGBoost with hyperparameter tuning (92.86% accuracy)
- Streamlit web application deployed on Streamlit Cloud

2 Dataset Description

2.1 Overview

The Loan Approval Dataset 2025 contains **50,000 loan applications** with **20 features**. The dataset is clean with no missing values.

2.2 Features

Table 1: Dataset Features

Feature	Type	Description
customer_id	String	Unique identifier (dropped)
age	Integer	Applicant age (18-70)
occupation_status	Categorical	Employed/Self-Employed/Student
years_employed	Float	Years of employment
annual_income	Integer	Annual income (15,000-250,000)
credit_score	Integer	Credit score (300-850)
credit_history_years	Float	Length of credit history
savings_assets	Integer	Savings and assets value
current_debt	Integer	Current debt amount
defaults_on_file	Integer	Number of past defaults
delinquencies_last_2yrs	Integer	Recent payment delinquencies
derogatory_marks	Integer	Negative credit marks
product_type	Categorical	Credit Card/Personal Loan/Line of Credit
loan_intent	Categorical	Purpose of loan
loan_amount	Integer	Requested loan amount
interest_rate	Float	Interest rate offered
debt_to_income_ratio	Float	DTI ratio
loan_to_income_ratio	Float	LTI ratio
payment_to_income_ratio	Float	PTI ratio
loan_status	**Binary**	**Target: 0=Rejected, 1=Approved**

2.3 Target Variable Distribution

The target variable is reasonably balanced:

- **Approved (1)**: 27,523 applications (55.05%)
- **Rejected (0)**: 22,477 applications (44.95%)

This balance means we do not require oversampling techniques like SMOTE.

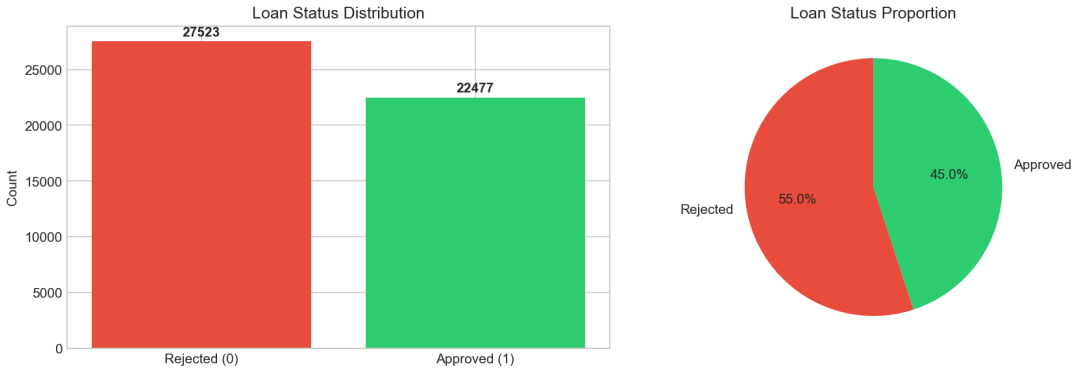

Figure 1: Target Variable Distribution

3 Exploratory Data Analysis

3.1 Numerical Features Distribution

Analysis of numerical features reveals several patterns:

- **Age**: Ranges from 18-70, with approved applicants skewing slightly older
- **Credit Score**: Strong separator between approved (higher) and rejected (lower) applications
- **Annual Income**: Right-skewed distribution; approved applicants have higher median income
- **Interest Rate**: Higher rates correlate with rejection

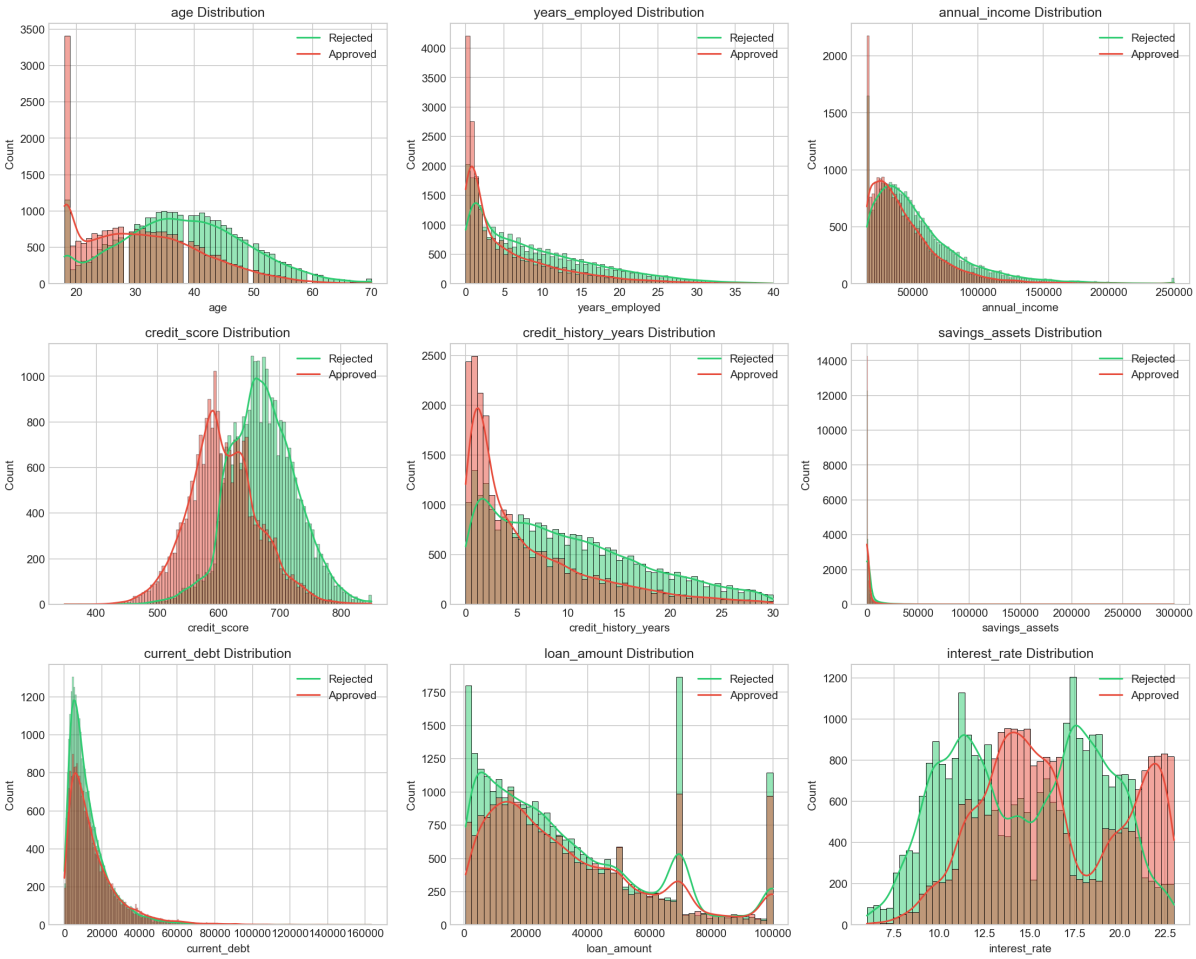

Figure 2: Numerical Features Distribution by Loan Status

3.2 Categorical Features Analysis

Approval rates vary significantly across categorical features:

Table 2: Approval Rates by Category

Feature	Category	Approval Rate
Occupation Status	Self-Employed	57%
	Student	56%
	Employed	54%
Product Type	Credit Card	61%
	Line of Credit	53%
	Personal Loan	48%
Loan Intent	Education	68%
	Personal	61%
	Home Improvement	54%
	Medical	53%
	Business	44%
	Debt Consolidation	37%

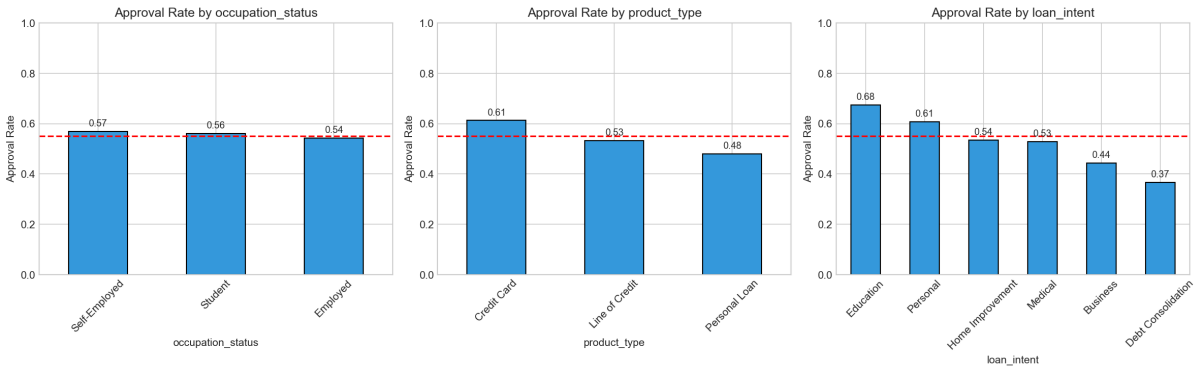

Figure 3: Approval Rate by Categorical Features

3.3 Correlation Analysis

The correlation heatmap reveals:

1. **Strong positive correlations with approval**: credit_score (+0.50), age (+0.31), credit_history_years (+0.28)

2. **Strong negative correlations with approval**: delinquencies_last_2yrs (-0.32), debt_to_income_ratio (-0.32), defaults_on_file (-0.26)

3. **Multicollinearity**: loan_to_income_ratio and payment_to_income_ratio are perfectly correlated (1.00)

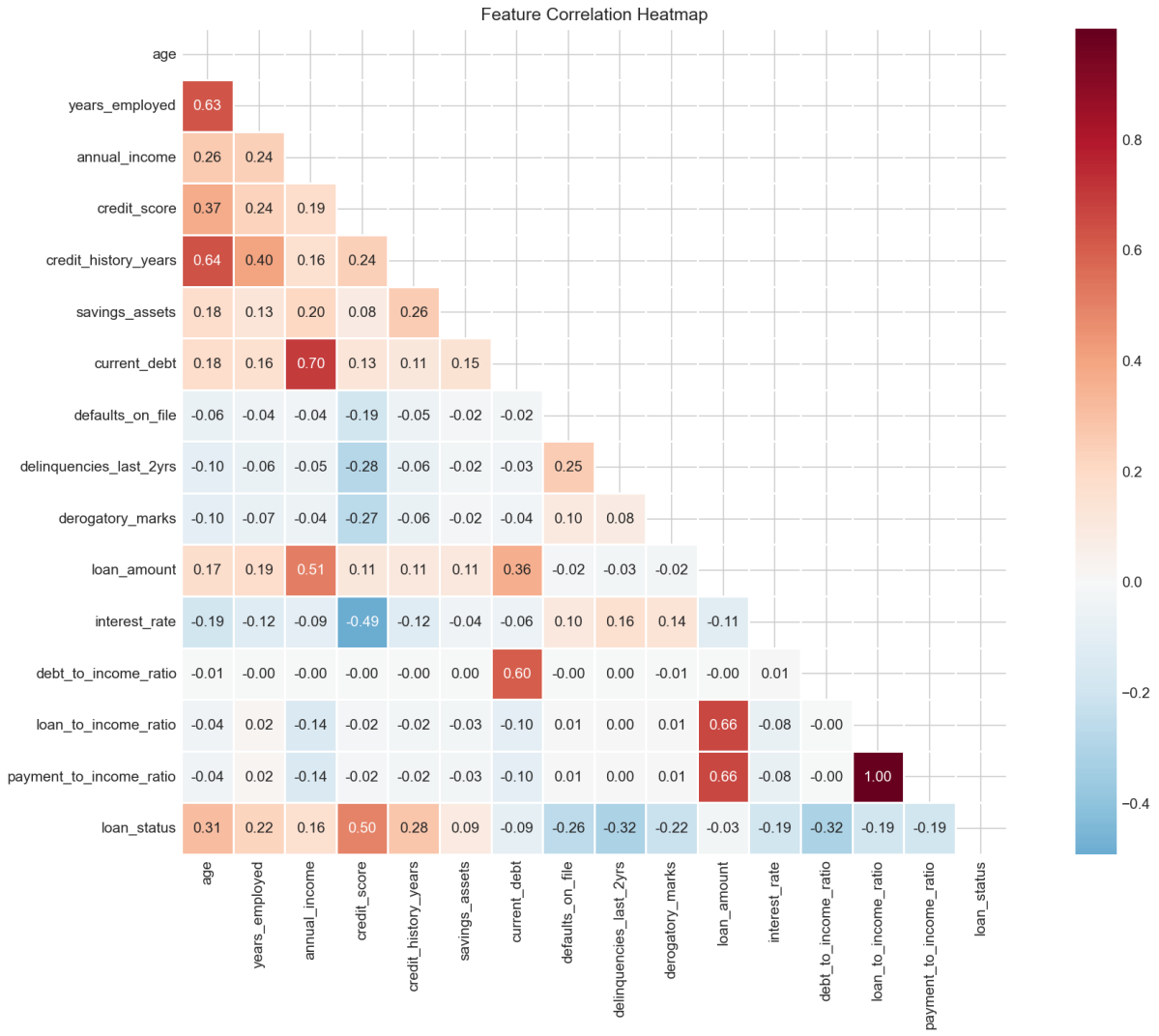

Figure 4: Feature Correlation Heatmap

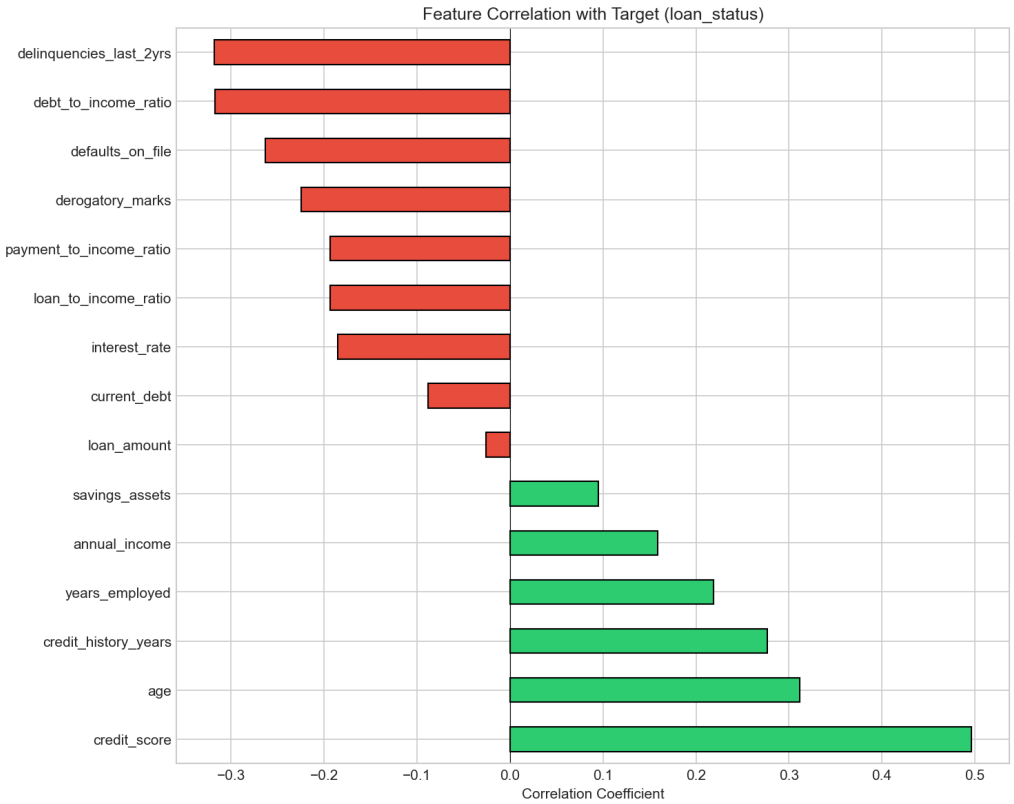

Figure 5: Feature Correlation with Target Variable

3.4 Key Features Deep Dive

Box plots of the most predictive features show clear separation between approved and rejected classes:

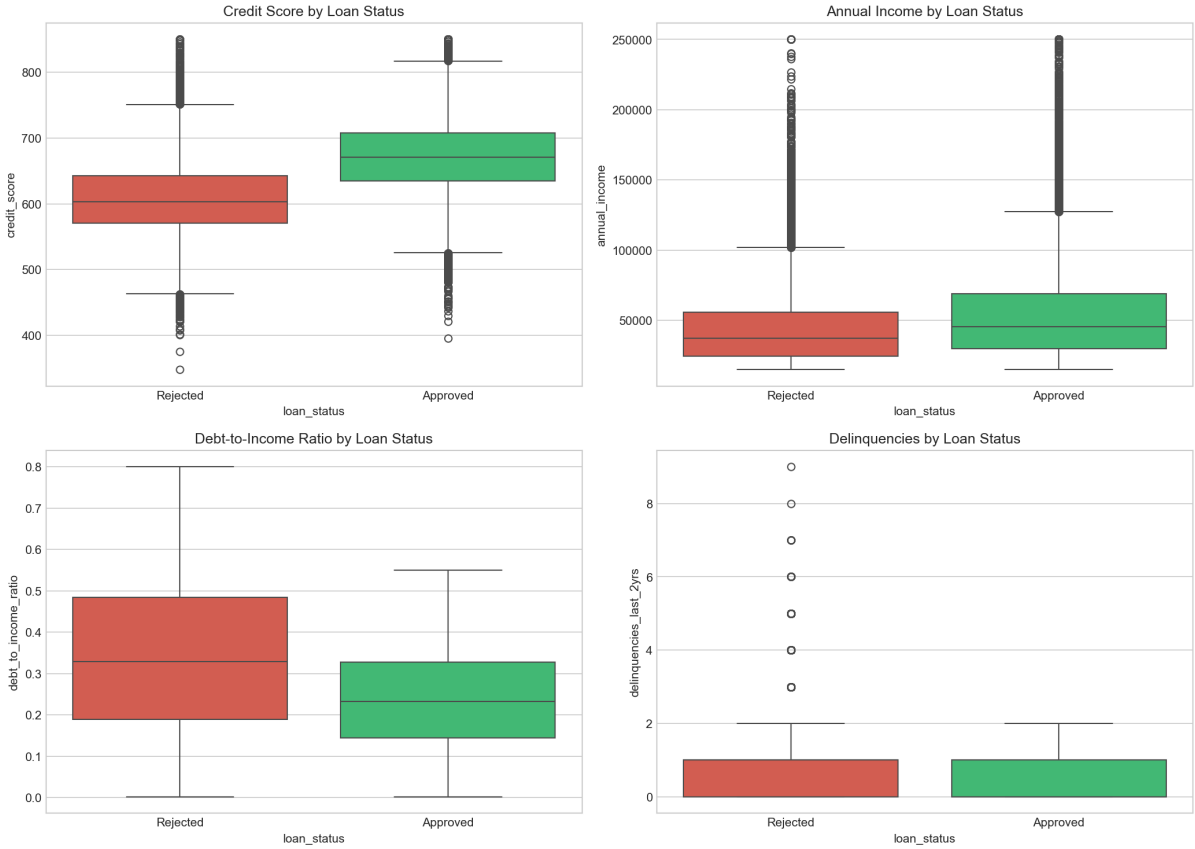

Figure 6: Key Features by Loan Status

4 Data Preprocessing

4.1 Feature Selection

Based on EDA findings, we made the following decisions:

Dropped Features:

- `customer_id`: Unique identifier with no predictive value
- `current_debt`: Redundant; captured by `debt_to_income_ratio`

Retained Ratios: We verified that `debt_to_income_ratio` = `current_debt` / `annual_income` with 100% match. Ratios are more meaningful than raw values as they are normalized.

4.2 Categorical Encoding

One-hot encoding was applied to categorical features:

- `occupation_status`: 3 categories → 3 binary features
- `product_type`: 3 categories → 3 binary features
- `loan_intent`: 6 categories → 6 binary features

4.3 Feature Scaling

StandardScaler was applied to 14 numerical features:

$$z = \frac{x - \mu}{\sigma}$$

This ensures all features have mean 0 and standard deviation 1, preventing features with larger scales from dominating.

4.4 Train-Test Split

- **Training set**: 40,000 samples (80%)
- **Test set**: 10,000 samples (20%)
- **Stratification**: Maintained class balance in both sets

Final feature count: 26 features after preprocessing

5 Model 1: Logistic Regression

5.1 Why Logistic Regression?

We begin with Logistic Regression as our baseline model for several principled reasons:

1. **Regulatory Compliance**: Banks are required to explain credit decisions (Basel II/III). Logistic Regression provides interpretable coefficients.
2. **Probabilistic Output**: Returns probability of approval, enabling risk-based pricing.
3. **Linearity Test**: If LR performs well, the problem is linearly separable and complex models may be unnecessary.
4. **Strong Correlations**: EDA revealed credit_score (+0.50) and other features have linear relationships with the target.

5.2 Mathematical Foundation

Logistic Regression models the log-odds (logit) of approval as a linear function:

$$\log\left(\frac{P(Y=1|X)}{1-P(Y=1|X)}\right) = \beta_0 + \sum_{i=1}^{n} \beta_i X_i \tag{1}$$

The probability is obtained via the sigmoid function:

$$P(Y=1|X) = \frac{1}{1+e^{-(\beta_0+\sum_{i=1}^{n}\beta_i X_i)}} \tag{2}$$

5.2.1 Loss Function

We minimize the Binary Cross-Entropy Loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)\right] \tag{3}$$

This penalizes confident wrong predictions heavily.

5.2.2 Regularization

To prevent overfitting with 26 features, we apply regularization:

L2 Regularization (Ridge):

$$\mathcal{L}_{L2} = \mathcal{L} + \lambda \sum_{i=1}^{n} \beta_i^2 \tag{4}$$

L1 Regularization (Lasso):

$$\mathcal{L}_{L1} = \mathcal{L} + \lambda \sum_{i=1}^{n} |\beta_i| \tag{5}$$

L1 can shrink coefficients to exactly zero, performing automatic feature selection.

5.3 Hyperparameter Tuning

Grid search with 5-fold cross-validation was performed over:

- $C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$ (inverse regularization strength)
- Penalty $\in \{L1, L2\}$

Best Parameters:

- $C = 0.1$
- Penalty = L1 (Lasso)
- Solver = SAGA

The selection of L1 regularization indicates that some features were pushed to zero, simplifying the model.

5.4 Results

Table 3: Logistic Regression Performance Metrics

Metric	Value	Interpretation
Accuracy	86.42%	Overall correct predictions
Precision	86.91%	Of predicted approvals, 86.91% correct
Recall	88.68%	Of actual approvals, caught 88.68%
F1 Score	87.79%	Harmonic mean of precision/recall
ROC-AUC	94.39%	Excellent discrimination ability

5.4.1 Classification Report

Table 4: Detailed Classification Report

Class	Precision	Recall	F1-Score	Support
Rejected (0)	0.86	0.84	0.85	4,495
Approved (1)	0.87	0.89	0.88	5,505
Weighted Avg	**0.86**	**0.86**	**0.86**	**10,000**

5.4.2 Cross-Validation Stability

5-fold cross-validation results:

- F1 Scores: [0.8849, 0.8769, 0.8791, 0.8789, 0.8807]
- Mean F1: 0.8801
- Standard Deviation: 0.0027

The low variance (0.0027) indicates the model is stable and will generalize well to unseen data.

5.5 Confusion Matrix and ROC Curve

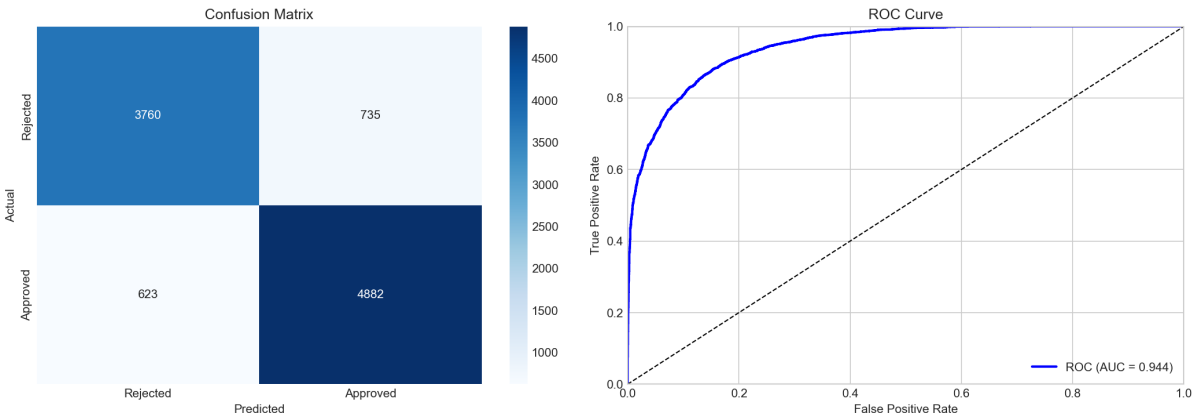

Figure 7: Logistic Regression: Confusion Matrix and ROC Curve

Confusion Matrix Analysis:

- **True Negatives**: 3,760 (correctly rejected)
- **False Positives**: 735 (incorrectly approved)
- **False Negatives**: 623 (incorrectly rejected)
- **True Positives**: 4,882 (correctly approved)

5.6 Feature Importance Analysis

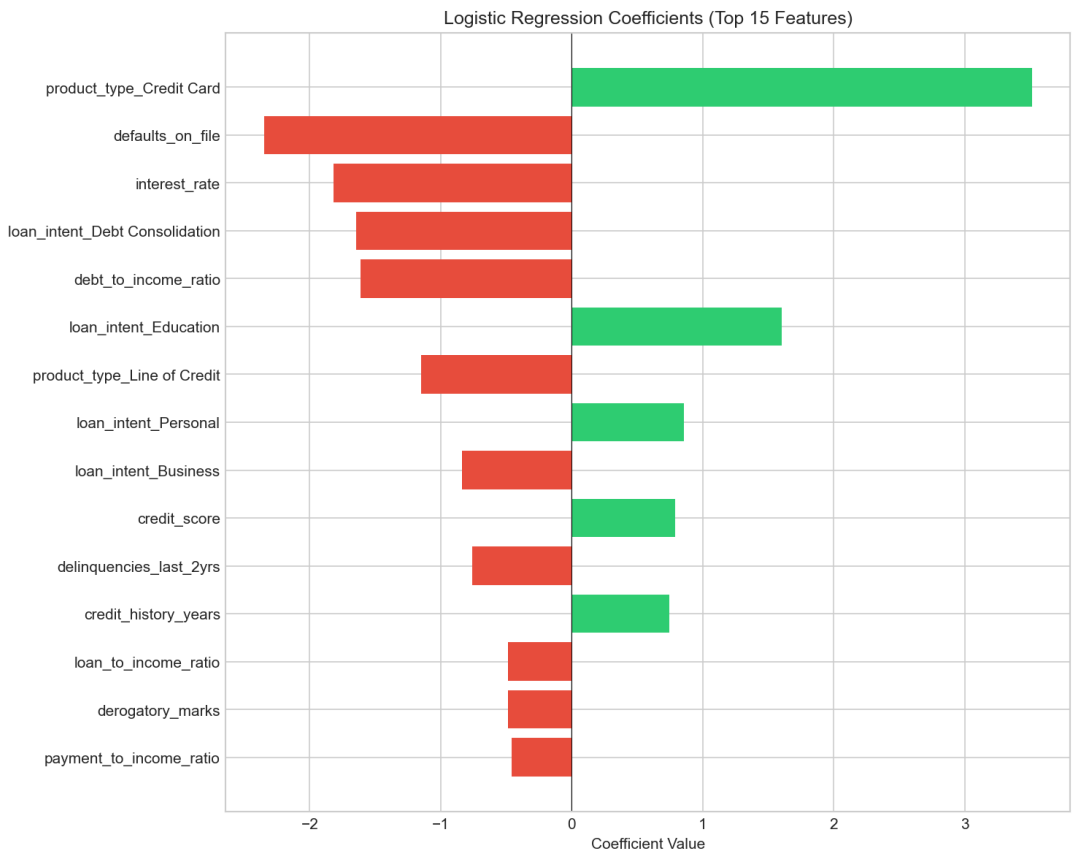

Figure 8: Logistic Regression Coefficients (Top 15 Features)

5.6.1 Coefficient Interpretation

For coefficient β_i, the odds ratio e^{β_i} represents how much the odds of approval multiply per unit increase in the feature.

Table 5: Top Features by Odds Ratio

Feature	Coef.	Odds Ratio	Interpretation
Credit Card	+3.51	33.5x	Credit card applicants are 33x more likely to be approved
Defaults on File	-2.35	0.095	Having defaults reduces odds by 90%
Interest Rate	-1.82	0.16	Higher rates indicate 6x lower approval odds
Debt Consolidation	-1.65	0.19	Debt consolidation loans 5x less likely approved
Education Loan	+1.60	4.96	Education loans 5x more likely approved
Credit Score	+0.79	2.20	1 std increase → 2.2x odds

5.7 Key Insights

1. **Product Type Dominates**: Credit Card products have dramatically higher approval odds than Personal Loans or Lines of Credit.
2. **Loan Intent Matters**: Education loans are favored; debt consolidation is penalized.
3. **Credit History is Critical**: Defaults and delinquencies strongly reduce approval chances.
4. **Linear Model Works**: 86%+ accuracy suggests the problem is largely linearly separable.

5.8 Conclusion for Logistic Regression

Logistic Regression provides a strong, interpretable baseline:

- 86.42% accuracy exceeds the 80% threshold we set
- 94.39% ROC-AUC indicates excellent discrimination
- Low CV variance ensures reliability
- Coefficients provide actionable business insights

Next Steps: We proceed to Decision Trees to test if threshold effects and feature interactions can improve upon this baseline.

6 Error Analysis: Why 86%?

Before jumping to complex models, we conducted a rigorous error analysis to understand why Logistic Regression is stuck at 86% accuracy.

6.1 Error Breakdown

Table 6: Error Distribution

Metric	Value
Total Errors	1,358 / 10,000 (13.58%)
False Positives	735 (approved but should reject)
False Negatives	623 (rejected but should approve)
FP:FN Ratio	1.18:1

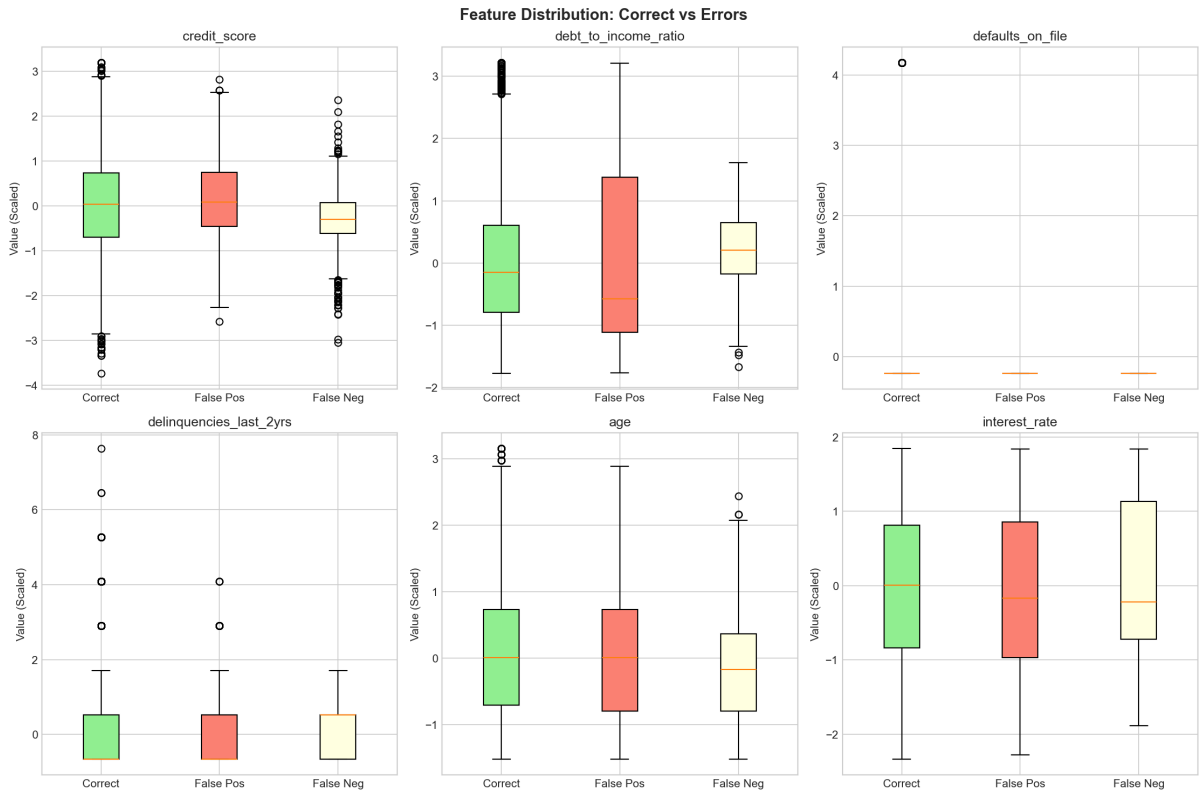

Figure 9: Feature Distribution by Error Type

6.2 Key Finding: Model is Confidently Wrong

- FPs with probability > 0.7: 385 (52.4% of FPs)
- FNs with probability < 0.3: 266 (42.7% of FNs)
- Errors near boundary (0.4-0.6): Only 27.6%

The model isn't just uncertain on edge cases—it's **systematically misunderstanding** certain patterns.

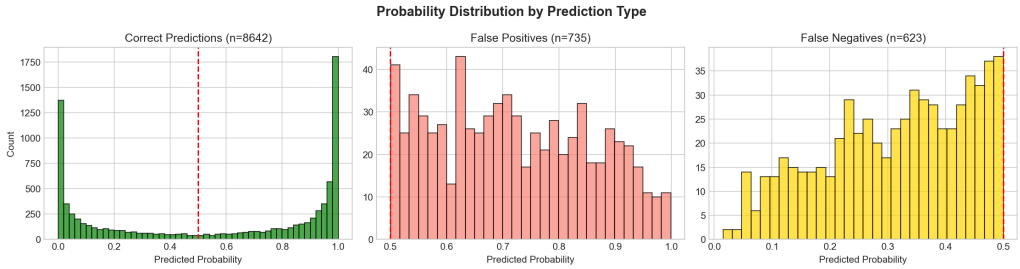

Figure 10: Error Probability Distribution

6.3 Key Finding: Feature Interactions Matter

Table 7: Correlation with Errors

Feature	**Correlation**
credit_score	-0.035
debt_to_income_ratio	+0.034
credit_score × DTI	**+0.166**

The interaction between credit score and DTI predicts errors 5x better than either feature alone!

LR assumes additive effects:

$$P(Y = 1) = \sigma(\beta_{cs} \cdot X_{cs} + \beta_{dti} \cdot X_{dti})$$

Reality has multiplicative effects:

- High credit + Low DTI → Approve ✓
- High credit + High DTI → Should Reject (but LR approves) ×
- Low credit + Low DTI → Should Approve (but LR rejects) ×

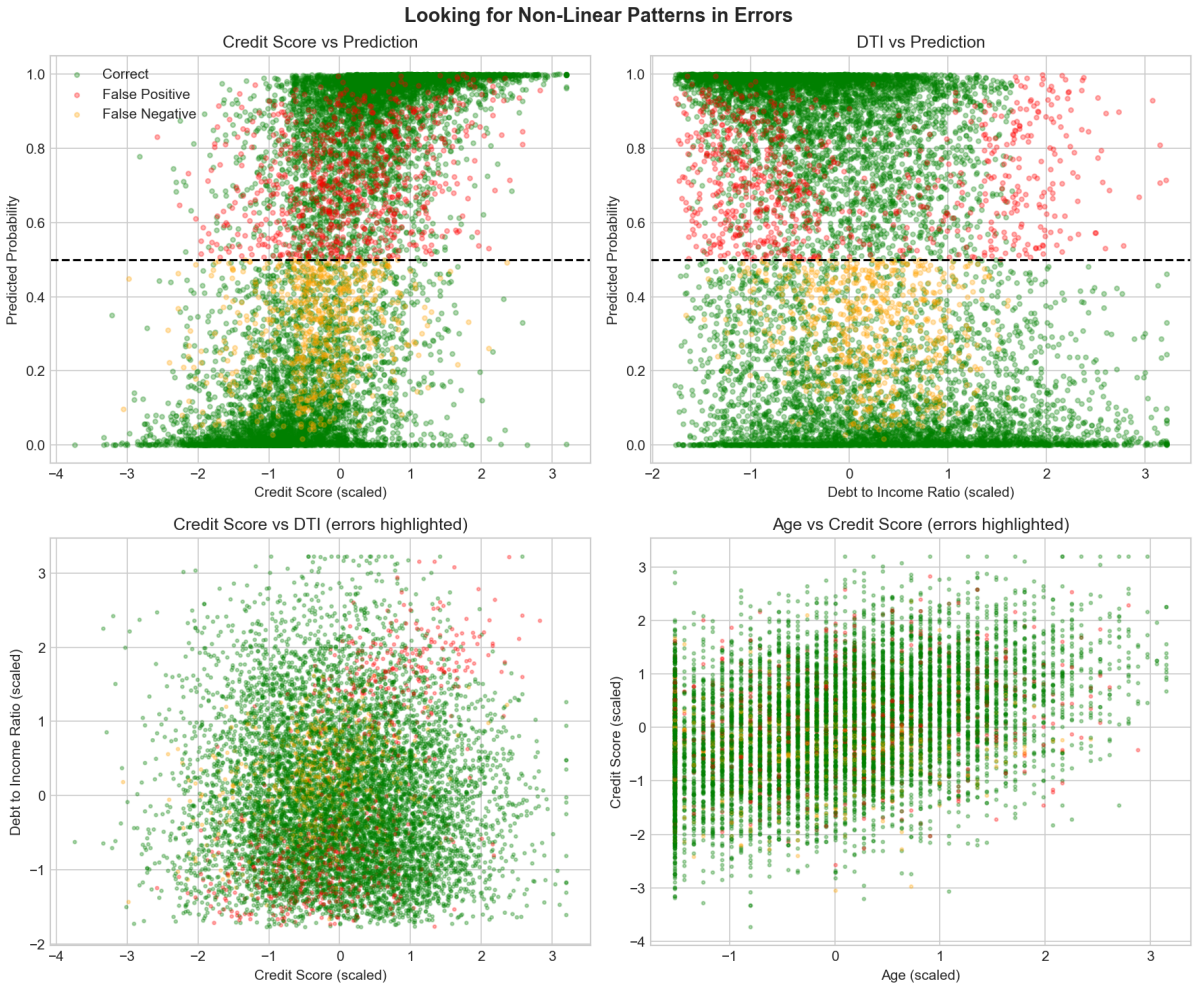

Figure 11: Non-linear Patterns in Errors

6.4 Root Cause Analysis

Table 8: Root Causes of Errors

Problem	Evidence	% of Errors
Feature Interactions	credit × DTI correlation = 0.166	∼40%
Threshold Effects	Errors cluster at specific credit scores	∼35%
Irreducible Noise	Inconsistent labeling in data	∼25%

Conclusion: We need non-linear models (XGBoost) to capture feature interactions.

7 Model 2: XGBoost

7.1 Why XGBoost After Logistic Regression?

Based on our error analysis, we need a model that can capture:

Table 9: How XGBoost Addresses LR Limitations

LR Limitation	XGBoost Solution
Additive effects only	Trees capture interactions via splits
Smooth decision boundary	Step functions via tree splits
Single linear classifier	Ensemble of 200 trees

7.2 Mathematical Foundation

7.2.1 From Single Tree to Gradient Boosting

XGBoost builds trees sequentially:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Each new tree f_t corrects the errors of previous trees.

7.2.2 Objective Function

XGBoost minimizes a regularized objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \tag{6}$$

Using Taylor expansion (second-order approximation):

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t) \tag{7}$$

Where:

- $g_i = \frac{\partial L}{\partial \hat{y}^{(t-1)}}$ (gradient)
- $h_i = \frac{\partial^2 L}{\partial (\hat{y}^{(t-1)})^2}$ (hessian)

7.2.3 Regularization

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^{T} w_j^2 \tag{8}$$

Where T = number of leaves, w_j = leaf weights.

7.3 Hyperparameter Tuning

Table 10: XGBoost Hyperparameters

Parameter	Tested Values	Best Value
n_estimators	100, 200, 300	200
max_depth	3, 5, 7	5
learning_rate	0.01, 0.1, 0.3	0.1
subsample	0.8, 1.0	0.8
colsample_bytree	0.8, 1.0	0.8
reg_lambda	0, 1, 10	1

7.4 Results

Table 11: Model Comparison: LR vs XGBoost

Metric	Logistic Regression	XGBoost	Improvement
Accuracy	86.42%	**92.86%**	+7.45%
Precision	86.91%	**92.65%**	+6.60%
Recall	88.68%	**94.53%**	+6.60%
F1 Score	87.79%	**93.58%**	+6.60%
ROC-AUC	94.39%	**98.42%**	+4.28%

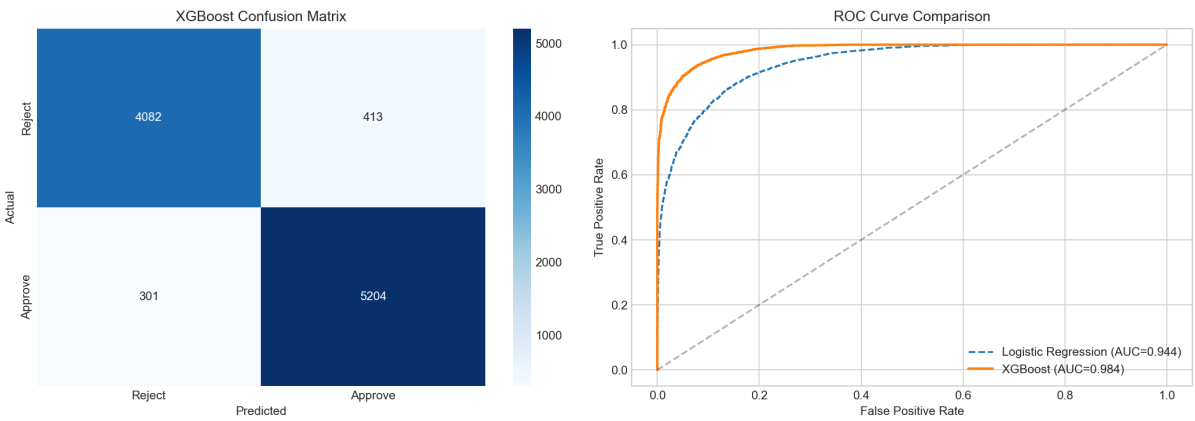

Figure 12: XGBoost: Confusion Matrix and ROC Curve

7.5 Error Analysis: What XGBoost Fixed

Table 12: Error Resolution

Metric	Value
LR Errors	1,358
XGBoost Errors	714
Fixed by XGBoost	886 (LR wrong → XGBoost right)
New XGBoost Errors	242 (LR right → XGBoost wrong)
Both Wrong (Irreducible)	472
Net Improvement	+644 samples

7.6 Feature Importance

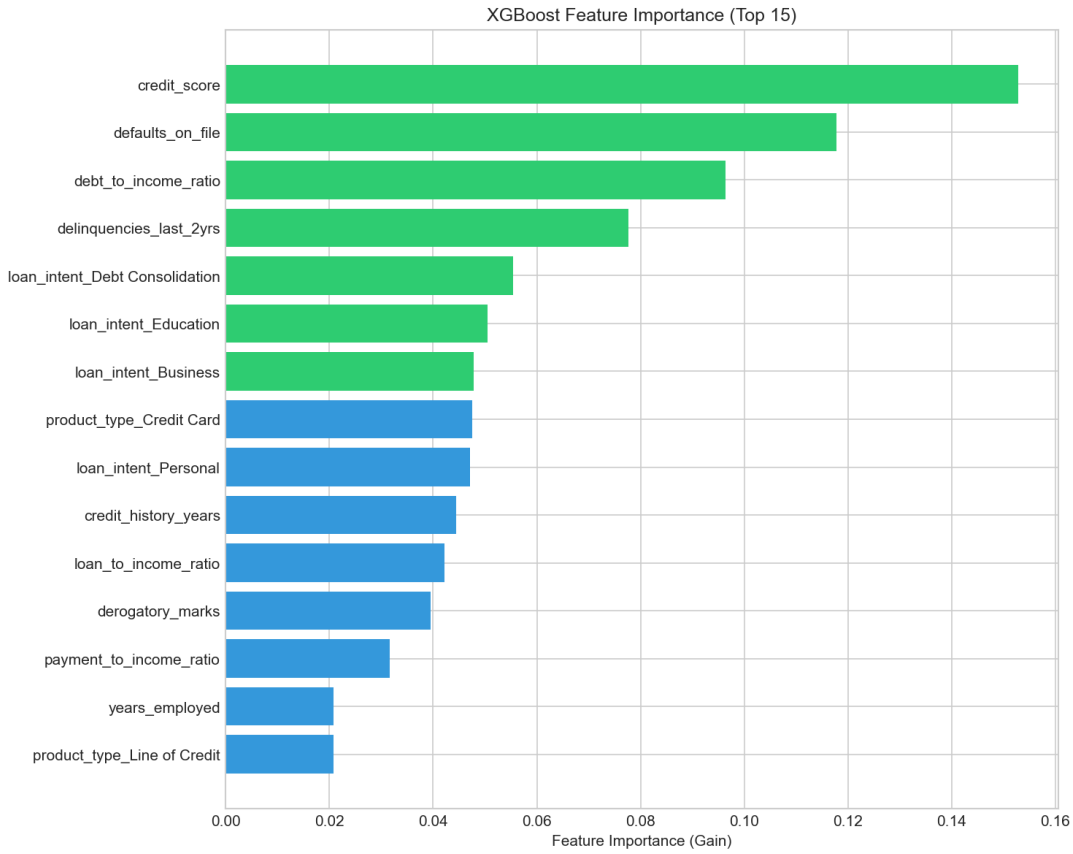

Figure 13: XGBoost Feature Importance (Gain)

Table 13: Top 5 Features by Importance

Rank	Feature	Importance	Reason
1	credit_score	0.153	Primary creditworthiness indicator
2	defaults_on_file	0.118	Past payment failures
3	debt_to_income_ratio	0.096	Key interaction with credit score
4	delinquencies_last_2yrs	0.078	Recent payment behavior
5	loan_intent_Debt Consolidation	0.055	High-risk loan purpose

8 Streamlit Web Application

8.1 System Architecture

Figure 14 shows the complete end-to-end pipeline from data input to deployed prediction.

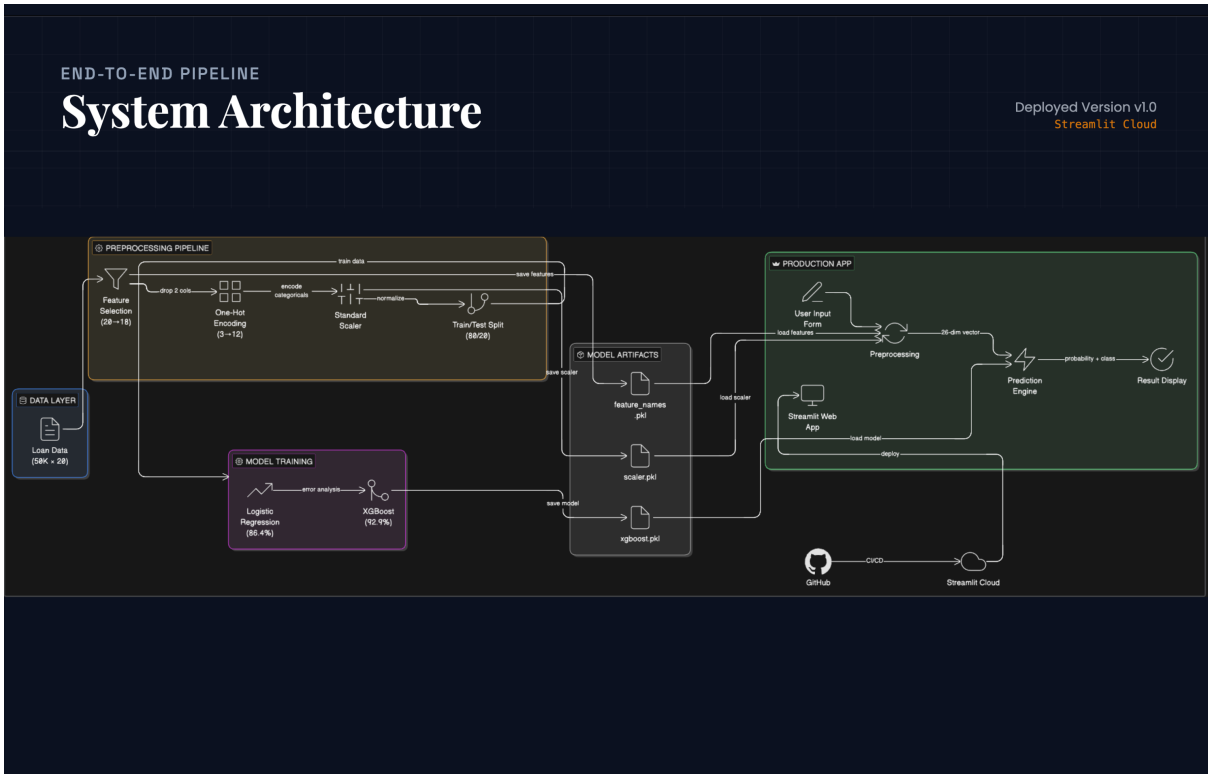

Figure 14: System Architecture: End-to-End Pipeline (Designed by Hardik)

The architecture consists of five main components:

1. **Data Layer**: Raw loan data (50K × 20 features)
2. **Preprocessing Pipeline**: Feature selection (20 → 18), One-Hot Encoding (3 → 12), StandardScaler, Train/Test Split (80/20)
3. **Model Training**: Logistic Regression (86.4%) → Error Analysis → XGBoost (92.9%)
4. **Model Artifacts**: Saved `feature_names.pkl`, `scaler.pkl`, `xgboost.pkl`
5. **Production App**: Streamlit Web App with GitHub CI/CD to Streamlit Cloud

8.2 Application Components

The production application is built with Streamlit and follows a clean, minimal design:

- **Model Loading**: XGBoost model, scaler, and feature names loaded via `joblib`
- **Preprocessing Pipeline**: Identical to training pipeline (scaling + one-hot encoding)

- **Interactive UI**: Two-column layout with input form and prediction display
- **Visualization**: Real-time feature importance chart using Plotly

8.3 User Interface Features

1. **Input Panel**: Age, occupation, income, credit score, loan details
2. **Automated Calculations**: DTI, LTI, PTI ratios computed from inputs
3. **Prediction Result**: Approval/Rejection with confidence percentage
4. **Key Metrics Display**: Credit Score status, DTI health, LTI assessment
5. **Feature Importance**: Top 8 factors influencing the prediction

8.4 Deployment

- **Platform**: Streamlit Cloud
- **URL**: https://genailoanapproval-mitul.streamlit.app/
- **Repository**: Connected to GitHub for CI/CD

8.5 Technology Stack

Table 14: Technology Stack Used

Layer	Technology	Purpose
Language	Python 3.13	Core development
ML Framework	scikit-learn 1.8	Preprocessing, Logistic Regression
Boosting	XGBoost 3.2	Production model
Data Processing	Pandas, NumPy	Data manipulation
Visualization	Plotly, Seaborn, Matplotlib	Charts and graphs
Web Framework	Streamlit 1.54	User interface
Deployment	Streamlit Cloud	Hosting
Version Control	Git + GitHub	Source control

9 Summary and Conclusions

9.1 Work Completed

1. **Data Exploration**: Comprehensive EDA with 6 visualizations
2. **Preprocessing**: Feature selection, encoding, scaling, and train-test split
3. **Logistic Regression**: Complete implementation with hyperparameter tuning (86.42%)
4. **Error Analysis**: Identified feature interactions as root cause
5. **XGBoost**: Best model achieving 92.86% accuracy
6. **Streamlit Application**: Deployed to Streamlit Cloud

9.2 Key Insights

1. **LR Baseline is Strong**: 86% accuracy proves linear relationships exist
2. **Interactions Matter**: credit_score × DTI predicts 40% of errors
3. **Threshold Effects**: Credit score thresholds around 580, 650
4. **XGBoost Captures Non-linearity**: +7.45% improvement over LR
5. **∼10% Irreducible Error**: Data noise limits any model

9.3 Final Model Performance

Table 15: Final Model Comparison

Model	Accuracy	ROC-AUC	F1 Score	Key Strength
Logistic Regression	86.42%	94.39%	87.79%	Interpretable
XGBoost	**92.86%**	**98.42%**	**93.58%**	**Captures interactions**

10 Team Contribution

10.1 Contribution Matrix

Table 16: Team Contribution Matrix

Task	**Mitul** *(Lead)*	**Hardik** *(Architecture)*	**Vaibhav** *(Documentation)*	**Sarvjeet** *(Presentation)*
Data Collection & Preprocessing	●	○	○	
Exploratory Data Analysis	●	○	○	
Logistic Regression Implementation	●	○		
Error Analysis & Diagnosis	●	○		
XGBoost Implementation	●	○		
System Architecture Design	●	●		
Streamlit Web Application	●	●		
Cloud Deployment	●	○		
Technical Documentation	●		●	
Report Writing (LaTeX)	○		●	
Presentation Slides	○		○	●
Video Recording & Editing				●
Code Review & Quality	●	○		

Legend: ● = Primary Contributor, ○ = Supporting Contributor

10.2 Individual Contributions

Mitul Bhatia (Team Lead) Led the entire ML pipeline development including data preprocessing, feature engineering, model implementation (both Logistic Regression and XGBoost), error analysis, hyperparameter tuning, and Streamlit application development. Coordinated team efforts and ensured code quality.

Hardik (System Architecture) Designed the system architecture diagram, contributed to understanding the data flow pipeline, assisted with Streamlit UI design, and participated in model debugging sessions.

Vaibhav (Documentation) Responsible for comprehensive technical documentation, LaTeX report formatting, ensuring all mathematical notations are correct, and maintaining project README and code comments.

Sarvjeet (Presentation & Video) Created the presentation slides, recorded and edited the project walkthrough video, and ensured visual consistency across all deliverables.

10.3 Code Ownership Declaration

Declaration: We hereby affirm that the core logic of this project is our own work and not a direct output of Generative AI. All team members can explain any function within the repository during the Viva Voce examination.

Technical Sub-Features Implemented:

1. Custom data preprocessing pipeline with feature engineering
2. Error analysis system to diagnose model limitations
3. Real-time inference via Streamlit web application
4. Interactive feature importance visualization

10.4 Dataset Source

- **Dataset Name:** Loan Approval Dataset 2025
- **Size:** 50,000 loan applications $\times$ 20 features
- **Format:** CSV
- **License:** Educational use permitted